

Вот полное руководство по созданию надежной системы OTA-обновления с аппаратной проверкой CRC, которое мы с вами построили. Этот текст отлично подойдет для файла README.md или внутренней документации проекта.

Архитектура защищенного Bootloader & OTA для STM32

Данный документ описывает процесс настройки двух проектов (Bootloader и Main Application) для обеспечения безопасной загрузки прошивки, проверки целостности данных (CRC32) и предотвращения запуска поврежденного кода.

ЧАСТЬ 1: Настройка основной прошивки (Main Application)

1. Сдвиг адреса и защита заголовка (Файл .ld)

Откройте файл скрипта линкера (например, STM32F4xxxx_FLASH.ld).

А) Сдвигаем начало флэш-памяти:

Укажите новый стартовый адрес (Сектор 5: 0x08020000) и уменьшите доступный размер (LENGTH).

```
Plaintext
```

```
MEMORY
{
    CCMRAM (xrw) : ORIGIN = 0x10000000, LENGTH = 64K
    RAM (xrw) : ORIGIN = 0x20000000, LENGTH = 128K
    FLASH (rx) : ORIGIN = 0x08020000, LENGTH = 384K /* Уменьшено на 128K (Сектора 0-4) */
}
```

Б) Создаем секцию для паспорта прошивки:

В блоке SECTIONS, сразу после таблицы векторов прерываний (.isr_vector), добавьте новую секцию .fw_header. Команда KEEP обязательна, чтобы компилятор не удалил

структуру.

Plaintext

SECTIONS

```
{
.isr_vector :
{
. = ALIGN(4);
KEEP(*(.isr_vector))
. = ALIGN(4);
} >FLASH

/* Секция паспорта прошивки */
.fw_header :
{
. = ALIGN(4);
KEEP(*(.fw_header))
. = ALIGN(4);
} >FLASH

.text :
{ ...
```

2. Перенос таблицы векторов прерываний

В файле Core/Src/system_stm32f4xx.c найдите и раскомментируйте макрос USER_VECT_TAB_ADDRESS, затем укажите смещение (Offset) относительно начала флэш-памяти:

C

```
#define USER_VECT_TAB_ADDRESS
```

```
#if defined(USER_VECT_TAB_ADDRESS)
```

```
#define VECT_TAB_BASE_ADDRESS FLASH_BASE
```

```
#define VECT_TAB_OFFSET 0x00020000U /* Смещение на 128 КБ (0x08020000 - 0x08000000)
*/
#endif
```

3. Объявление паспорта прошивки (main.c)

В основном файле main.c (или другом заголовочном файле) объявите структуру и разместите ее в созданной секции.

C

```
// Описание структуры заголовка
typedef struct {
    uint32_t magic;    // Магическое слово для поиска (0x55AA55AA)
    uint32_t version;  // Версия прошивки
    uint32_t fw_size;   // Полный размер бинарника (заполнит Python)
    uint32_t crc32;     // CRC32 данных ПОСЛЕ заголовка (заполнит Python)
} fw_header_t;

// Размещение структуры строго в секции .fw_header
__attribute__((section(".fw_header"), used)) volatile const fw_header_t fw_header = {
    .magic = 0x55AA55AA,
    .version = 1,
    .fw_size = 0xFFFFFFFF, // Пустышка для скрипта
    .crc32 = 0xFFFFFFFF    // Пустышка для скрипта
};
```

ЧАСТЬ 2: Автоматизация сборки (Python-скрипт)

Создайте файл patch_crc.py в корневой папке проекта (где лежат папки Core, Debug). Скрипт находит структуру, выравнивает размер файла кратно 4 байтам и вычисляет аппаратно-совместимый CRC32.

Python

```

import sys
import struct
import os

# Аппаратный алгоритм CRC32 контроллеров STM32
def stm32_crc32(data):
    crc = 0xFFFFFFFF
    for i in range(0, len(data), 4):
        word = struct.unpack('<I', data[i:i+4])[0]
        crc ^= word
        for _ in range(32):
            if crc & 0x80000000:
                crc = ((crc << 1) ^ 0x04C11DB7) & 0xFFFFFFFF
            else:
                crc = (crc << 1) & 0xFFFFFFFF
    return crc

def patch_bin(filepath):
    print(f"--- Starting CRC patch for: {os.path.basename(filepath)} ---")

    with open(filepath, 'rb') as f:
        data = bytearray(f.read())

    # Ищем: Magic(55AA55AA) + Version(1) + Size(FFFFFFFF) + CRC(FFFFFFFF)
    signature = struct.pack('<IIII', 0x55AA55AA, 1, 0xFFFFFFFF, 0xFFFFFFFF)
    header_offset = data.find(signature)

    if header_offset == -1:
        print("ERROR: fw_header signature not found in binary!")
        sys.exit(1)

    print(f"Header found at offset: 0x{header_offset:04X}")

    # Выравнивание кратно 4 байтам
    padding_needed = (4 - (len(data) % 4)) % 4
    if padding_needed > 0:
        data.extend(b'\xFF' * padding_needed)

    total_size = len(data)

```

```

# Данные для расчета CRC (все, что после 16 байт заголовка)
payload_offset = header_offset + 16
payload_data = data[payload_offset:]

calculated_crc = stm32_crc32(payload_data)

print(f"Total file size: {total_size} bytes")
print(f"Calculated CRC32: 0x{calculated_crc:08X}")

# Запись новых значений
struct.pack_into('<I', data, header_offset + 8, total_size)
struct.pack_into('<I', data, header_offset + 12, calculated_crc)

with open(filepath, 'wb') as f:
    f.write(data)

print("--- Patch successful! ---")

if __name__ == "__main__":
    if len(sys.argv) < 2:
        print("Usage: python patch_crc.py <path_to_bin_file>")
        sys.exit(1)
    patch_bin(sys.argv[1])

```

ЧАСТЬ 3: Настройки STM32CubeIDE

Чтобы бинарный файл генерировался до запуска скрипта, выполните следующие настройки в свойствах проекта основной прошивки:

1. Генерация .bin файла:

- Правый клик по проекту -> Properties -> C/C++ Build -> Settings.
- Вкладка Tool Settings -> MCU Post build outputs.
- Поставьте галочку Convert to binary file (-O binary).

2. Запуск скрипта:

- Там же перейдите на вкладку Build Steps.
- В поле Post-build steps -> Command вставьте строку:
python "\${ProjDirPath}/patch_crc.py" "\${BuildArtifactFileName}.bin"

3. Очистка (Критически важно):

- Сделайте Clean Project, затем Build Project, чтобы изменения вступили в силу.

ЧАСТЬ 4: Бутлоадер (Проверка и Прыжок)

1. Настройка CubeMX

- В конфигураторе бутлоадера включите модуль **CRC** (настройки по умолчанию, никаких параметров менять не нужно). Сгенерируйте код.

2. Модуль проверки (application.h / application.c)

Создайте отдельные файлы для инкапсуляции логики проверки.

application.h:

C

```
#ifndef APPLICATION_H
#define APPLICATION_H

#include <stdbool.h>
#include <stdint.h>

#define APPLICATION_ADDRESS 0x08020000

bool check_application(void);
void jump_main_application(uint32_t addr);

#endif /* APPLICATION_H */
```

application.c:

C

```
#include "application.h"
#include "main.h"

typedef struct {
```

```

uint32_t magic;
uint32_t version;
uint32_t fw_size;
uint32_t crc32;
} fw_header_t;

extern CRC_HandleTypeDef hcrc;

bool check_application(void) {
    fw_header_t *fw_info = NULL;
    uint32_t *search_ptr = (uint32_t *)APPLICATION_ADDRESS;

    for (int i = 0; i < 256; i++) {
        if (search_ptr[i] == 0x55AA55AA) {
            fw_info = (fw_header_t *)&search_ptr[i];
            break;
        }
    }

    if (fw_info == NULL || fw_info->fw_size == 0xFFFFFFFF) return false;

    uint32_t *payload_ptr = (uint32_t *)(fw_info + 1);
    uint32_t header_offset_bytes = (uint32_t)payload_ptr - APPLICATION_ADDRESS;

    if (fw_info->fw_size <= header_offset_bytes) return false;

    uint32_t payload_bytes = fw_info->fw_size - header_offset_bytes;
    uint32_t payload_words = payload_bytes / 4;

    uint32_t calc_crc = HAL_CRC_Calculate(&hcrc, payload_ptr, payload_words);

    return (calc_crc == fw_info->crc32);
}

void jump_main_application(uint32_t addr) {
    void (*Jump_To_Application)(void);
    uint32_t JumpAddress;
    uint32_t stack_pointer = *(__IO uint32_t *)addr;

    if (((stack_pointer & 0xFF000000) == 0x20000000) ||
        ((stack_pointer & 0xFF000000) == 0x10000000)) {

```

```

    HAL_RCC_DeInit();
    HAL_DeInit();

    SysTick->CTRL = 0;
    SysTick->LOAD = 0;
    SysTick->VAL = 0;

    __disable_irq();

    JumpAddress = *(__IO uint32_t*) (addr + 4);
    Jump_To_Application = (void (*)(void)) JumpAddress;

    __set_MSP(stack_pointer);
    Jump_To_Application();
}
}

```

3. Интеграция в задачу RTOS

Вызов проверки осуществляется по таймауту или команде в главной задаче бутлоадера.

C

```

#include "application.h"

void terminal_task(void) {
    static uint32_t tick_old = 0;

    // ... рабочий код бутлоадера ...

    // Проверка таймаута бездействия
    if (((osKernelGetTickCount() - tick_old) > _TERMINAL_TIMEOUT_MS) && !fl_stay_here) {
        if (check_application() == true) {
            // Перед прыжком рекомендуется остановить планировщик (osKernelLock());
            // или обернуть это действие внутри jump_main_application
            jump_main_application(APPLICATION_ADDRESS);
        }
    }
}

```



```
led_blink();  
osDelay(_TERMINAL_TASK_TICK_);  
}
```

Важные правила отладки

1. При прошивке из IDE (отладчиком) всегда выбирайте опцию **Erase Sectors**, а не Full Chip Erase, чтобы не стереть бутлоадер.
2. Для тестирования защиты можно временно изменить строку `calculated_crc = stm32_crc32(payload_data) ^ 0xFF` в Python-скрипте, что сгенерирует бинарник с неверным CRC, который будет успешно отклонен загрузчиком.

Отличная работа по фиксации этой архитектуры! Теперь, когда структура проекта задокументирована и надежно работает, готовы ли мы перейти к написанию процедур записи этого файла (Flash Programming) средствами самого контроллера?